

```
>>> operator.le(10, 20)      # "Less than or Equal", same as 10 <= 20
True
```

The `operator` module gives us function versions of the comparison operators. Their implementations are simple. For example, we could write our own `operator.eq()` function in two lines:

```
def eq(a, b):
    return a == b
```

It's useful to have a function form of the comparison operators because, unlike operators, functions can be passed as arguments to function calls. We'll be doing this to implement a helper method for our comparison dunder methods.

First, add the following to the start of *wizcoin.py*. These imports give us access to the functions in the `operator` module and allow us to check whether the `other` argument in our method is a sequence by comparing it to `collections.abc.Sequence`:

```
import collections.abc
import operator
```

Then add the following to the end of the *wizcoin.py* file:

```
--snip--
❶ def _comparisonOperatorHelper(self, operatorFunc, other):
    """A helper method for our comparison dunder methods."""

❷     if isinstance(other, WizCoin):
        return operatorFunc(self.total, other.total)
❸     elif isinstance(other, (int, float)):
        return operatorFunc(self.total, other)
❹     elif isinstance(other, collections.abc.Sequence):
        otherValue = (other[0] * 17 * 29) + (other[1] * 29) + other[2]
        return operatorFunc(self.total, otherValue)
        elif operatorFunc == operator.eq:
            return False
        elif operatorFunc == operator.ne:
            return True
    else:
        return NotImplemented
```